

Portfolio Question 4.1.P - Constrained Optimisation

Daniel Lawson University of Bristol

Portfolio 4.1.P (v4.0.0)

Portfolio Question 4.1.P - Constrained Optimisation

- ▶ How is Constrained Optimisation related to Eigenvalues and Eigenvectors?
 - ▶ Is this trick relevant to wider machine learning?

Leveling with Lagrange: An Alternate View of Constrained Optimization

DAN KALMAN

American University
Washington, D.C. 20016
kalman@american.edu

In most calculus books today [11, 14, 15], Lagrange multipliers are explained as follows. Say that we wish to find the maximum value of f subject to the condition that $g = 0$. Under certain assumptions about f and g , the Lagrange multipliers theorem asserts that at the solution point, the gradient vectors ∇f and ∇g are parallel. Therefore, either $\nabla f = \lambda \nabla g$ for some real number λ , or $\nabla g = 0$. Combined with the equation $g = 0$, this gives necessary conditions for a solution to the constrained optimization problem. We will refer to this as the standard approach to Lagrange multipliers.

An earlier tradition approaches this subject far differently. It defines a new function, $F = f + \lambda g$, that incorporates both the objective function and the constraint, and in which λ is considered to be an additional variable. Here, F is referred to as a *Lagrangian function*. The conditions for F to achieve an unconstrained extremum are then determined, and these become necessary conditions for a solution to the original problem. This is the Lagrangian function approach to Lagrange multipliers.

Minimisation problem

- ▶ Goal is to minimise

$$\mathbf{u}^T \mathbf{c} \mathbf{u}$$

- ▶ subject to $\mathbf{u}^t \mathbf{u} = 1$
- ▶ Lagranian approach says: add λ times the constraint
 - ▶ and solve for the minima in terms of both $\mathbf{u}$ and λ

Optimisation

- ▶ For simplicity we'll just add the k -th dimension, conditioning on orthogonality to the previous $k - 1$
 - ▶ This is how many algorithms work in practice
- ▶ Constraint: $\mathbf{u}^T \mathbf{u} = 1$. Therefore

$$\mathbb{L}(\mathbf{u}, \lambda) = \mathbf{u}^T \mathbf{c} \mathbf{u} - \lambda(\mathbf{u}^T \mathbf{u} - 1) \quad (1)$$

$$\frac{\partial \mathbb{L}}{\partial \lambda} = \mathbf{u}^T \mathbf{u} - 1 \quad (2)$$

$$\frac{\partial \mathbb{L}}{\partial \mathbf{u}} = 2\mathbf{c} \mathbf{u} - 2\lambda \mathbf{u} \quad (3)$$

- ▶ Which can be solved for when the derivatives are zero to get

$$\mathbf{u}^T \mathbf{u} = 1 \quad (4)$$

$$\mathbf{c} \mathbf{u} = \lambda \mathbf{u} \quad (5)$$

Reflection

- ▶ This is the claimed Eigenvalue expression
- ▶ If we replace C by $C - \mathbf{u}^T C \mathbf{u}$ in the above, we are looking for the largest direction of variation perpendicular to the previously found Eigenspace
- ▶ Looking more widely we see entire workshops dedicated to **Constrained Optimisation for Machine Learning**
- ▶ These look technical
- ▶ One paper about **fairness** emphasises that this trick is about imposing **hard** constraints
 - ▶ i.e. exact restrictions like above (norm is exactly 1, directions are exactly orthogonal)
 - ▶ How is this different from soft constraints, e.g. “discouraging solutions that are far from the desired constraint”?

Discussion

- ▶ Is it computationally easy to impose hard constraints like this?
- ▶ How do we do it in other cases?
 - ▶ What happens if the constraint is impossible to satisfy?
- ▶ How would we impose soft constraints?
- ▶ When would we choose either approach?